Day 82: The Backward Pass Through the Whole Transformer, and a Gradient Check

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 12, Day 6 | Complete study guide

Watch: Backpropagation calculus, Deep Learning Chapter 4 (3Blue1Brown) | The spelled-out intro to neural networks and backpropagation: building micrograd (Andrej Karpathy)

Script: `day82_transformer_backward.py` · every number in this guide is copied by program from `day82_run_output.txt`, the real output of that script.

Training the Transformer needs the gradient of the loss for every weight. Each block's backward pass already exists; today they are chained, from the loss back to the shared embedding table, and checked numerically: 2714 entries of a model with two blocks per side and padded inputs, every one passing. A deliberate bug shows the check is sharp: dropping one decoder block's share of the encoder's gradient fails exactly the parts behind it and none of the others. And the gradient teaches: plain SGD fits a fixed batch of 32 REVERSE examples to a loss of 0.000.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

The **backward pass** applies the chain rule from the loss down through every layer, in reverse order of the forward pass. Two places need care in a Transformer. The **encoder's output** is read by every decoder block's cross-attention, so its gradient is the **sum** of what each sends back. The **embedding table** is used for both the source and the target, so its gradient is the sum of both sides.

REAL-WORLD SCENARIO

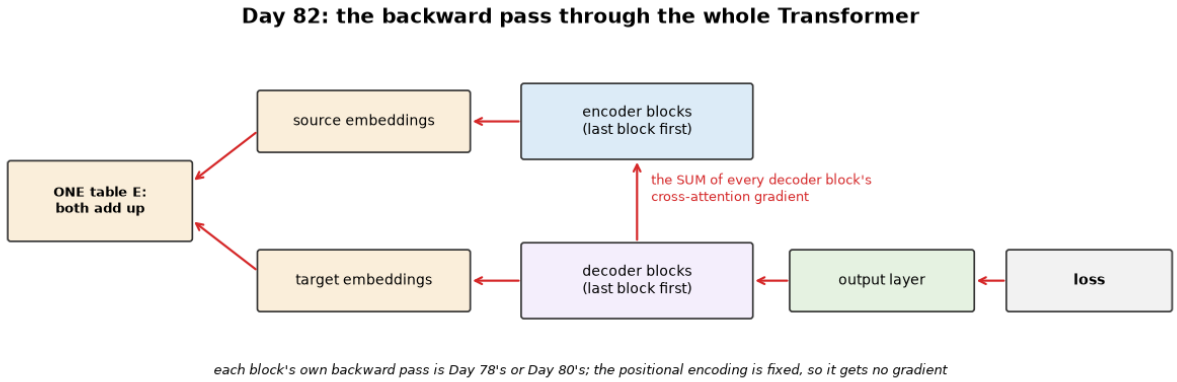
Tracing a fault back through a production line: the final product is checked first, then each station before it, and a part used by two stations has to answer for both.

MATHEMATICS

$$dL/d\,enc = \Sigma_{\text{decoder blocks } j}\; dL/d\,enc_{\text{(through block } j)}$$

$$dL/dE[t] = \Sigma_{\text{target positions with token } t}\; dL/dx + \Sigma_{\text{source positions with token } t}\; dL/dx$$

SYSTEM DIAGRAM



The backward pass, right to left.

Today's part for Day 83

`day82_exercise.py` adds `decoder_block_backward` and `transformer_backward` to the exercise model and checks every entry: Week 12's sixth and last part.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The order of the backward pass

DEFINITION — start here, no prior knowledge assumed

One backward pass for 32 COPY examples, with the gradient's size at each hand-over point, from the output layer down to the embedding table.

REAL-WORLD SCENARIO

Following the blame back through a relay team, from the last runner to the first.

MATHEMATICS

$$dlogits = (softmax - one\text{-}hot) / (real\ positions);\quad dW_{out} = dec^{T}\, dlogits;\quad ddec = dlogits\, W_{out}^{T}$$

IMPLEMENTATION

```
def gradients(self, y, broken=False):
    m, T = self.tgt_in.shape
    mask = self.tmask.ravel()
    dlog = np.exp(self.logp); dlog[np.arange(y.size), y.ravel()] -= 1
    dlog *= mask[:, None] / mask.sum()
    G = dict(Wout=self.dec_out.reshape(-1, D).T @ dlog, bout=dlog.sum(axis=0))
    dy = (dlog @ self.P["Wout"].T).reshape(m, T, D)
    denc = np.zeros_like(self.enc_out)
    for i in reversed(range(len(self.dec))):
        dy, de, g = self.dec[i].backward(dy)
        denc = de if broken else denc + de
        G.update({f"d{i}.{k}": v for k, v in g.items()})
    G["E"] = np.zeros_like(self.P["E"])
    np.add.at(G["E"], self.tgt_in, dy)
    dx = denc
    for i in reversed(range(len(self.enc))):
        dx, g = self.enc[i].backward(dx)
        G.update({f"e{i}.{k}": v for k, v in g.items()})
    np.add.at(G["E"], self.src, dx)
    return G
```

ACTUAL OUTPUT

```
stage                                                shape       size
output layer Wout                                 (32, 10)   5.88e-01
d0.W2                                             (64, 32)   4.37e-02
d0.cq                                             (32, 32)   7.46e-03
d0.ck                                             (32, 32)   6.79e-03
d0.sq                                             (32, 32)   1.10e-02
e0.W2                                             (64, 32)   3.30e-02
e0.Wq                                             (32, 32)   6.11e-03
E                                                 (10, 32)   7.97e-03
(d0 = the decoder block, e0 = the encoder block; cq, ck: cross-attention's query and key weights;
    sq: self-attention's)
Actual output from this lesson's run.
```

What it means

The output layer's gradient is the largest (5.88e-01); the blocks' are around 10^{-2} and the embedding table's 7.97e-03. The encoder's gradient (e0) arrives only through the decoder's cross-attention, which is why the chain must pass through it.

WHY CHOOSE THIS?

Reuses every block's own checked backward pass.

WHY NOT THIS? (TRADEOFFS)

Order matters: one swapped step and the check fails.

2. Gradient check of the whole model

DEFINITION — start here, no prior knowledge assumed

Every entry of every parameter, nudged both ways, for a tiny model with two encoder and two decoder blocks, padded source and target positions, and LayerNorm weights moved off 1 and 0.

REAL-WORLD SCENARIO

Testing every circuit of a two-storey building with the doors in the unused rooms shut.

SPECIFICATION

Model	d = 8, 2 heads, FFN 12, two blocks per side, output layer × 0.1 (Day 81).
Data	3 examples; source padded in two of them, target in one; Day 62's pass rule.

Why this test

Two blocks per side test the parts one block cannot: the encoder gradient summed over decoder blocks, and blocks stacked on blocks.

IMPLEMENTATION

```
def gradient_check(net, batch, broken=False, eps=1e-5):
    src, sm, tgt_in, tm, y = batch
    net.forward(src, sm, tgt_in, tm); G = net.gradients(y, broken)
    groups = {}
    for k, A in net.params().items():
        worst, ok = 0.0, True
        for pos in np.ndindex(A.shape):
            old = A[pos]
            A[pos] = old + eps; up = net.loss(src, sm, tgt_in, tm, y)
            A[pos] = old - eps; dn = net.loss(src, sm, tgt_in, tm, y)
            A[pos] = old
            num = (up - dn) / (2 * eps)
            worst = max(worst, abs(num - G[k][pos])); ok &= abs(num - G[k][pos]) <= 1e-10 + 1e-6 *
                abs(num)
        g = k if k in ("E", "Wout", "bout") else k.split(".")[0]
        w, o = groups.get(g, (0.0, True)); groups[g] = (max(w, worst), o and ok)
    return groups
```

ACTUAL OUTPUT

```
2714 entries checked
part                         largest |numeric - analytic|  every entry passes
embedding table (shared)                          3.79e-11               True
output layer W                                    6.73e-11               True
output layer b                                    6.31e-11               True
encoder block 1                                   5.98e-11               True
encoder block 2                                   6.88e-11               True
decoder block 1                                   6.02e-11               True
decoder block 2                                   6.22e-11               True
Actual output from this lesson's run.
```

What it means

All 2714 entries pass, the largest difference being 6.88e-11. The backward pass is right for the whole model, including the shared table and the summed encoder gradient.

WHY CHOOSE THIS?

A complete, entry-by-entry check of the model that will be trained.

WHY NOT THIS? (TRADEOFFS)

Tiny sizes; the real model runs the same code.

3. Would the check catch a bug?

DEFINITION — start here, no prior knowledge assumed

One deliberate mistake: the encoder's gradient keeps only the share from decoder block 1 and drops block 2's.

REAL-WORLD SCENARIO

Testing a smoke detector by holding a lit match under it.

Why this test

A check that cannot fail is worthless. This bug is a realistic one: overwriting a sum instead of adding to it.

IMPLEMENTATION

```
denc = de if broken else denc + de
```

ACTUAL OUTPUT

```
part                          largest |numeric - analytic|  every entry passes
embedding table (shared)                        4.69e-03               False
output layer W                                  6.73e-11                True
output layer b                                  6.31e-11                True
encoder block 1                                 9.85e-03               False
encoder block 2                                 1.05e-02               False
decoder block 1                                 6.02e-11                True
decoder block 2                                 6.22e-11                True
Actual output from this lesson's run.
```

What it means

The check fails exactly where the bug's effect reaches: both encoder blocks (9.85e-03 and 1.05e-02) and the embedding table (4.69e-03), which collects from the encoder. Everything computed before the mistake in the backward pass, the output layer and both decoder blocks, still passes. The check does not just say 'wrong'; it says where.

WHY CHOOSE THIS?

Localises the bug to the parts downstream of it.

WHY NOT THIS? (TRADEOFFS)

Only as good as the batch: a bug on a path the batch never uses would not show.

4. Where the gradient goes, before training

DEFINITION — start here, no prior knowledge assumed

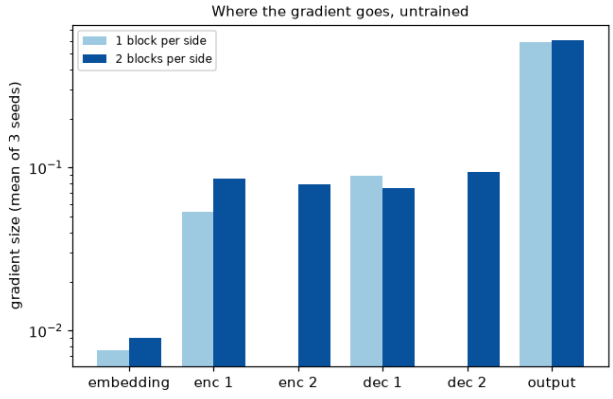
The size of each part's gradient for one batch, untrained, with one and with two blocks per side, three seeds.

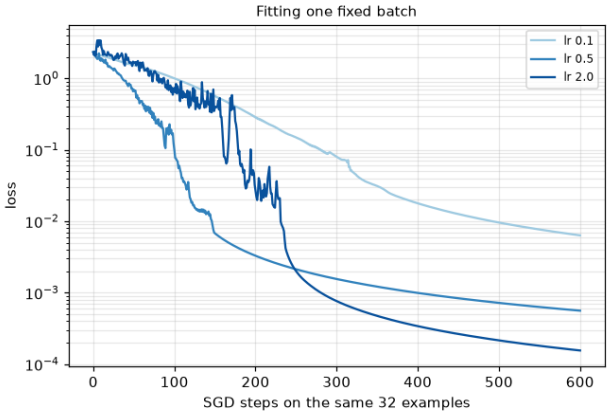
REAL-WORLD SCENARIO

Who gets the most feedback after the first rehearsal.

ACTUAL OUTPUT

```
 blocks seed   embedding       enc 1       enc 2       dec 1       dec 2      output
      1    0    7.97e-03    5.46e-02           -    8.84e-02           -    6.01e-01
      1    1    8.14e-03    6.57e-02           -    9.28e-02           -    5.72e-01
      1    2    6.60e-03    4.14e-02           -    8.60e-02           -    5.88e-01
      2    0    9.36e-03    7.83e-02    8.27e-02    7.30e-02    1.01e-01    6.17e-01
      2    1    8.02e-03    7.94e-02    7.17e-02    8.22e-02    9.79e-02    6.23e-01
      2    2    9.74e-03    1.01e-01    8.38e-02    7.00e-02    8.18e-02    5.72e-01
Actual output from this lesson's run.
```





Left: gradient size by part, mean of three seeds. Right: loss while fitting one fixed batch (Section 5).

What it means

The output layer gets by far the largest gradient (about 0.6), the blocks about a tenth of it, the embedding table about a hundredth (7.97e-03 to 9.74e-03). With two blocks per side the lower blocks get about as much as the upper ones: in this untrained model the gradient does not fade with depth (Day 78's finding: LayerNorm keeps its size).

WHY CHOOSE THIS?

Shows which parts will move first in training.

WHY NOT THIS? (TRADEOFFS)

Untrained only; the picture changes as training proceeds.

5. Does the gradient teach?

DEFINITION — start here, no prior knowledge assumed

Plain SGD on one fixed batch of 32 REVERSE examples for 600 steps, at three learning rates. A correct gradient should drive the loss towards 0 on data it sees again and again.

REAL-WORLD SCENARIO

Rehearsing one song until it is note-perfect: a test of the practice method, not of musicianship.

Why this test

A gradient can be correct and still useless if something else is wrong (the loss, the update, the scaling). Memorising one batch is the simplest end-to-end test.

IMPLEMENTATION

```
def train_step(self, src, smask, tgt_in, tmask, y, lr, clip=1.0):
    self.forward(src, smask, tgt_in, tmask)
    G, P = self.gradients(y), self.params()
    norm = np.sqrt(sum(np.sum(v ** 2) for v in G.values()))
    for k, v in G.items():
        P[k] -= lr * min(1.0, clip / norm) * v
    return norm
```

ACTUAL OUTPUT

```
    lr       @0      @50     @100     @200     @400     @600  token acc
   0.1    2.327    1.532    1.001    0.287    0.018    0.006      1.000
   0.5    2.327    0.693    0.099    0.003    0.001    0.001      1.000
   2.0    2.327    1.635    0.733    0.027    0.000    0.000      1.000
Actual output from this lesson's run.
```

What it means

All three rates reach token accuracy 1.000 on the batch. lr 0.5 is fastest early (0.099 after 100 steps); lr 2.0 is noisier but ends lowest (0.000); lr 0.1 is slow but steady (0.006 at 600). The model and its gradient can learn; whether it generalises to new sequences is Day 83's question.

WHY CHOOSE THIS?

A fast end-to-end test of forward, backward and update together.

WHY NOT THIS? (TRADEOFFS)

Memorising one batch says nothing about generalisation.

6. What the backward pass costs

ACTUAL OUTPUT

```
forward 3.45 ms; forward + backward 7.98 ms; the backward pass alone is 1.31 times the forward
    pass
Actual output from this lesson's run.
```

In this run the backward pass took 1.31 times as long as the forward pass: the same matrix products again, about once more. Only ratios within one run are meaningful.

Interview preparation — technical questions

Q: In what order does the backward pass visit the blocks?

A: Output layer, decoder blocks last to first, then encoder blocks last to first, then the embeddings.

Q: Why is the encoder's gradient a sum?

A: Every decoder block's cross-attention reads the same encoder output.

Q: What gradient does a shared embedding table get?

A: The sum of the source-side and target-side contributions.

Q: Does the positional encoding get a gradient?

A: No: the sinusoidal table is fixed.

Q: How do you check a whole model's gradient?

A: Nudge every entry both ways and compare with the analytic value (2714 entries here, all passing).

Interview preparation — theoretical questions

Q: Why test the gradient check with a deliberate bug?

A: To know it can fail; here it failed exactly in the parts downstream of the bug.

Q: Why fit one batch?

A: If the model cannot memorise 32 examples, something besides the gradient is broken.

Q: Why does the output layer get the largest gradient at the start?

A: It sits next to the loss; every other part is reached through it and through normalised layers.

Q: Why is the backward pass about as expensive as the forward pass?

A: It repeats the same matrix products, with the gradient in place of the input.

Key takeaway and what's next

KEY TAKEAWAY

The Transformer's backward pass is the blocks' own backward passes chained in reverse, with two sums: over the decoder blocks for the encoder's gradient, and over both sides for the shared embedding table. Checked entry by entry it is right; with one sum turned into an overwrite the check fails exactly where it should; and plain SGD with it fits a batch perfectly.

- Gradient check: 2714 entries, two blocks per side, largest difference 6.88e-11, all pass.
- The deliberate bug: encoder blocks and embedding table fail (up to 1.05e-02); output layer and decoder pass.
- Untrained gradient sizes: output about 0.6, blocks about 0.04-0.1, embeddings about 0.008.
- One fixed batch: token accuracy 1.000 at lr 0.1, 0.5 and 2.0.

Suggested watch order, if starting from zero

- 1. Backpropagation calculus, Deep Learning Chapter 4 (3Blue1Brown) — the chain rule through layers, visually.
- 2. The spelled-out intro to neural networks and backpropagation: building micrograd (Andrej Karpathy) — building backpropagation by hand, step by step.

Day 83 preview

- **The week's build.** A tiny Transformer assembled from Week 12's six exercise scripts, trained on COPY and REVERSE, and compared with Week 11's attention LSTM.
- Today's exercise script, `day82_exercise.py`, is the backward pass of the exercise model: the sixth and last part.